

# 挑战点一：点亮板载LED并使之闪烁

## 第一步：查询评估板手册

(一)、Nucleo板上的LED有三个，分别为：

- 红色的LD1（ST-LINK指示灯）
- 绿色的LD2（用户操作，高电平点亮、低电平熄灭）
- 橙色的LD3（电源指示灯）

(二)、从手册中可以看到LD2连接的引脚标志为**D13**，可能为PA5或PB13

### 7.6 LEDs

The tricolor LED (green, orange, red) LD1 (COM) provides information about ST-LINK communication status. LD1 default color is red. LD1 turns to green to indicate that communication is in progress between the PC and the ST-LINK/V2-1, with the following setup:

- Slow blinking Red/Off: at power-on before USB initialization
- Fast blinking Red/Off: after the first correct communication between the PC and ST-LINK/V2-1 (enumeration)
- Red LED On: when the initialization between the PC and ST-LINK/V2-1 is complete
- Green LED On: after a successful target communication initialization
- Blinking Red/Green: during communication with the target
- Green On: communication finished and successful
- Orange On: Communication failure

**User LD2:** the green LED is a user LED connected to ARDUINO® signal D13 corresponding to STM32 I/O PA5 (pin 21) or PB13 (pin 34) depending on the STM32 target. Refer to [Table 11](#) to [Table 23](#) when:

- the I/O is HIGH value, the LED is on
- the I/O is LOW, the LED is off

**LD3 PWR:** the red LED indicates that the STM32 part is powered and +5V power is available.

(三)、进一步查询Table11，得知LD2连接的引脚为**PA5**

**Table 11. ARDUINO® connectors on NUCLEO-F030R8, NUCLEO-F070RB, NUCLEO-F072RB, NUCLEO-F091RC (continued)**

| Connector   | Pin | Pin name | STM32 pin | Function               |
|-------------|-----|----------|-----------|------------------------|
| CN5 digital | 6   | D13      | PA5       | SPI1_SCK               |
|             | 5   | D12      | PA6       | SPI1_MISO              |
|             | 4   | D11      | PA7       | TIM17_CH1 or SPI1_MOSI |
|             | 3   | D10      | PB6       | TIM16_CH1N or SPI1_CS  |
|             | 2   | D9       | PC7       | TIM3_CH2               |
|             | 1   | D8       | PA9       | -                      |

结合以上信息，得出使用汇编语言点亮Nucleo的板载LED并使之闪烁的步骤：

RCC开启GPIOA的时钟->配置PA5引脚为推挽输出模式->在程序中设置死循环，使PA5引脚每隔相同时间在高低电平之间不断转换

## 第二步：确定编写的代码内容

### 第一部分：向量表段

- 1、使用段伪指令(AREA)声明这一段代码为只读数据段，用于存放向量表
- 2、声明函数调用、临时数据存放的内存空间的**栈顶地址**
- 3、声明硬件开始执行程序程序的**复位入口的标签地址**
- 4、若干指向同一个异常死循环的**异常向量**

### 第二部分：代码段声明

- 1、使用段伪指令(AREA)指出可执行程序的代码段
- 2、使用导出伪指令(EXPORT)公开复位入口标签

### 第三部分：复位入口程序

- 1、开启GPIOA外设时钟
- 2、配置PA5引脚为通用推挽输出模式
- 3、跳转到主循环标签

### 第四部分：主循环

- 1、配置BSRR寄存器将PA5电平置为高电平
- 2、调用延时子程序
- 3、配置BSRR寄存器将PA5电平置为低电平
- 4、调用延时子程序

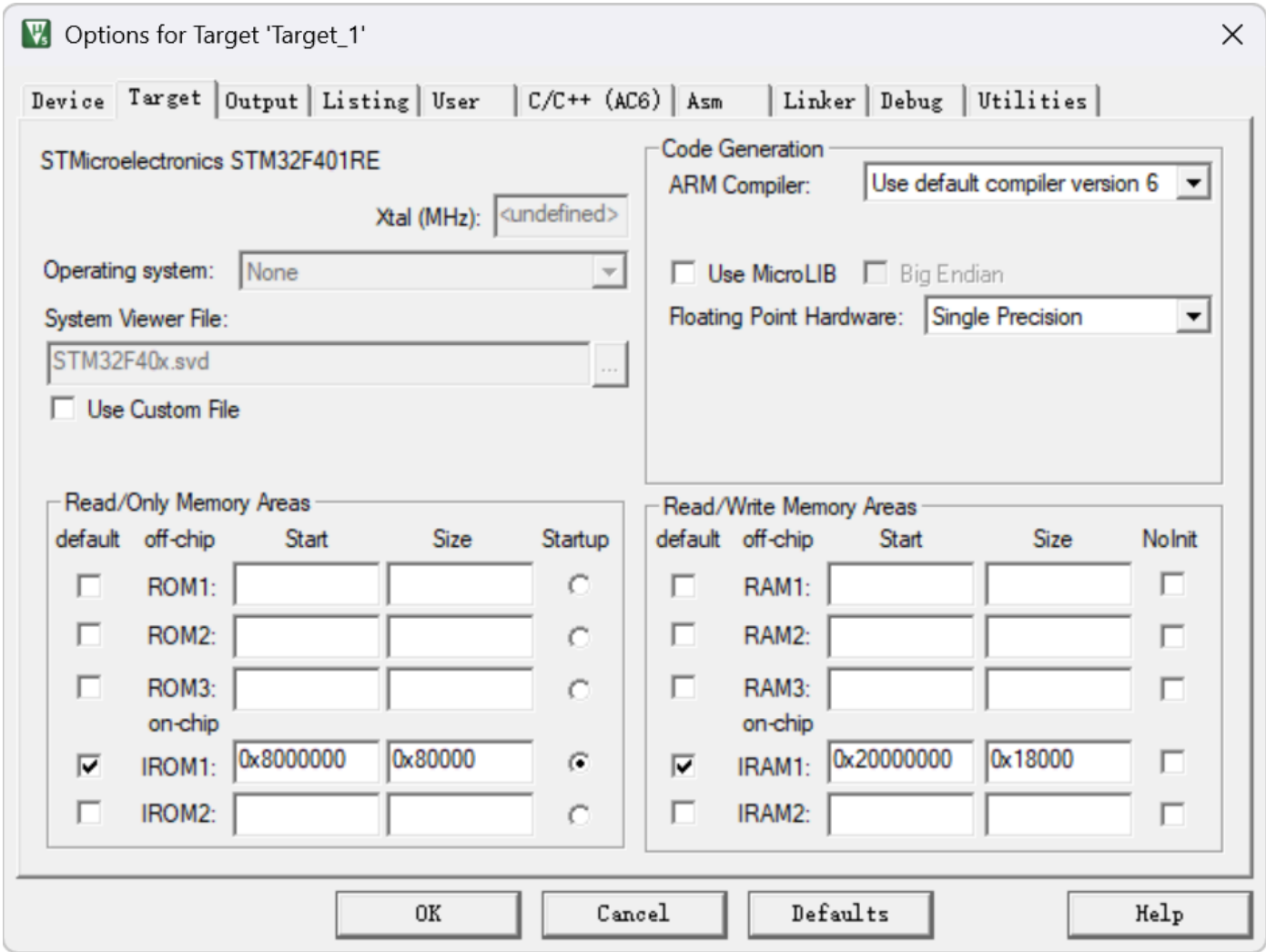
## 第五部分：子程序集合

- 1、软件延时子程序
- 2、异常死循环子程序

## 第三步：创建Keil项目并进行配置

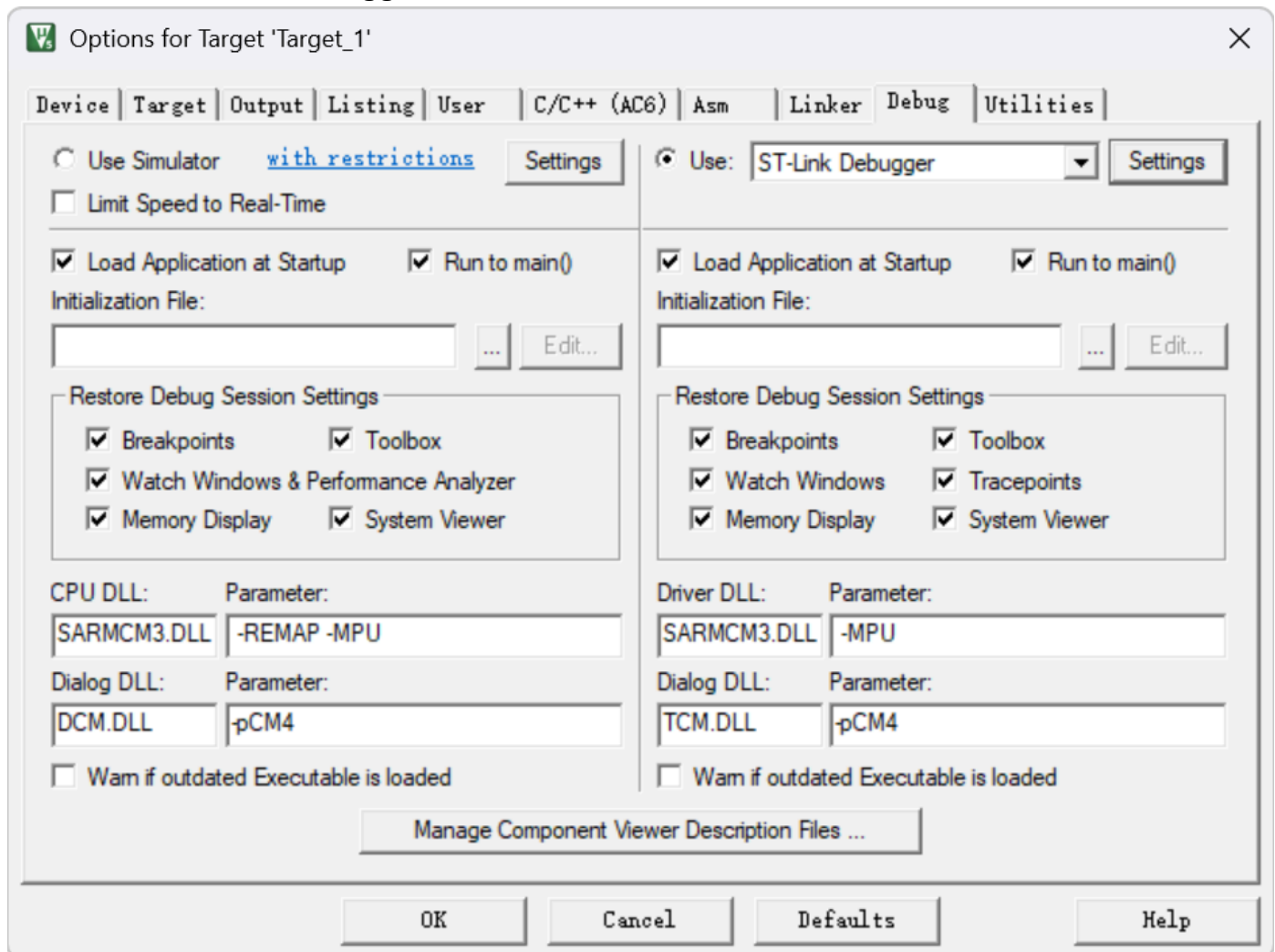
现在我们已经清楚了使用汇编语言点亮板载LED并使之闪烁的大致步骤，接下来就可以创建一个Keil项目来具体编写实现这个功能的程序：

- 1、创建一个新的Keil项目后选择STM32F401RE型芯片，然后点击cancel，不下载任何库文件
- 2、点击魔术棒，在target栏进行配置
  - **IROM (Internal Read-Only Memory/片内只读存储器)**：芯片内部的Flash，用于存放代码、常量字符串，数据断电不丢失。Size为512KB (0x80000)、地址起点0x8000000
  - **IRAM (Internal Random-Access Memory/片内随机读写内存)**：芯片的运行内存，用于存放局部变量、全局变量、栈、堆，数据断电丢失。Size为96KB (0x18000)、地址起点0x20000000

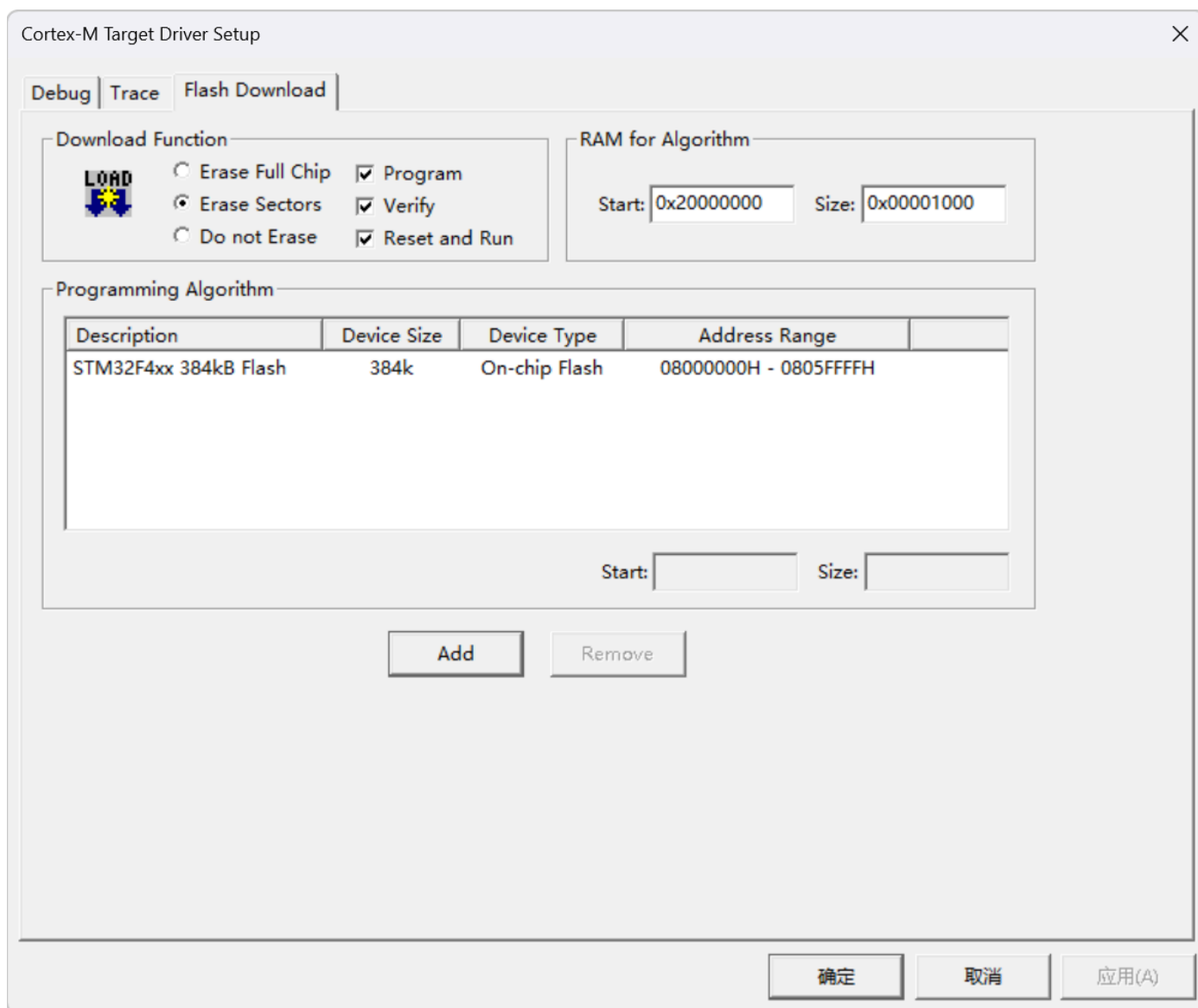


### 3、在Debug栏进行配置

- **Use:** 处选择 **ST-Link Debugger**，表示使用ST-Link下载和调试程序



- 点击ST-Link Debugger右侧的Settings，点击Flash Download栏，勾选Reset and Run，点击Add、OK。使之后每次下载程序后自动复位并执行程序



#### 4、创建main.s文件

展开Target1文件夹，右键Source Group 1点击Add New Item添加新文件，选择.s文件并命名为main，接下来在main.s文件中使用汇编语言编写需要下载到芯片中的代码

## 第四步：汇编语言学习

**伪指令：**写给汇编器/链接器的提示命令，只在编译阶段生效

**CPU指令：**写给CPU执行的机器命令

### (一)、CPU寄存器

STM32F401RET6的CPU内核使用的是Cortex-M4，一共有16个CPU寄存器（R0~R15）

- **通用寄存器（R0~R12）**：共13个，32位宽度，没有硬件专门分配固定用途，汇编程序员可自由使用
- **Stack Pointer/栈指针（R13）<sup>[1]</sup>**：指向栈的栈顶地址。数据入栈时SP的值减小

- **Link Register/链接寄存器 (R14)**: 保存子程序返回地址
- **Program Counter/程序计数器 (R15)**: 保存CPU要读取执行的指令地址
- **xPSR(程序状态寄存器)**: 保存比较指令CMP产生的标志位: 相等、不等于、零、负数等

## (二)、ARM汇编语言指令集合

- **伪指令:**

| 指令     | 使用示例                          | 功能说明                        |
|--------|-------------------------------|-----------------------------|
| AREA   | AREA CODE_SEG, CODE, READONLY | 定义段; 设置段类型 (CODE/DATA)、读写属性 |
| DCD    | DCD 0x20000000                | 在当前段存放 32 位字常量; 可填数字或标签名    |
| EXPORT | EXPORT Reset_Handler          | 把标签对外导出, 允许别的段引用该标签         |

- **CPU指令:**

| 指令       | 使用示例                     | 功能说明                              |
|----------|--------------------------|-----------------------------------|
| LDR      | LDR R0, [R1]             | 内存加载, 示例把R1指向的内存 / 外设寄存器的数据读取到R0中 |
| STR      | STR R0, [R1]             | 内存存储, 示例把R0的数据写入R1指向的内存 / 外设寄存器中  |
| ORR      | ORR R0, R0, #<br>(1<<5)  | 按位或, 示例将R0的 bit5 置 1, 其余位不变       |
| AND      | AND R0, R0, #~<br>(1<<3) | 按位与, 示例将R0的 bit3 清零, 其余位不变        |
| B        | B main_loop              | 无条件跳转, 不保存返回地址                    |
| BL       | BL delay_sub             | 带链接跳转 (子程序调用), 返回地址存入 LR          |
| BX<br>LR | BX LR                    | 子程序返回, 跳回 LR 保存的返回地址              |
| CMP      | CMP R2, #0               | 比较两个操作数, 更新状态标志位为第一个操作数减第二个操作数的值  |
| BNE      | BNE delay_inner          | 状态标志位不为0则跳转 (条件跳转)                |

# 第五步：确认需要操作的寄存器

| 寄存器名称         | 全称             | 本项目用途                        |
|---------------|----------------|------------------------------|
| RCC_AHB1ENR   | AHB1 外设时钟使能寄存器 | 开启 GPIOA 外设时钟                |
| GPIOA_MODER   | GPIO 模式寄存器     | 配置 PA5 为通用输出模式               |
| GPIOA_OTYPER  | GPIO 输出类型寄存器   | 配置 PA5 为推挽输出                 |
| GPIOA_OSPEEDR | GPIO 输出速度寄存器   | 配置 PA5 为低速输出                 |
| GPIOA_PUPDR   | GPIO 上拉下拉寄存器   | 配置 PA5 无上下拉电阻                |
| GPIOA_BSRR    | 置位 / 复位寄存器     | 控制 PA5 输出高 / 低电平，点亮 / 熄灭 LED |
| GPIOA_ODR     | GPIO 输出数据寄存器   | 调试观察引脚真实输出电平                 |

# 第六步：代码编写

编写代码时通过查阅 rm0368 stm32f401 参考手册查看寄存器地址和配置寄存器的位

PRESERVE8

THUMB

AREA RESET, DATA, READONLY ; 开启向量表段

DCD 0x20018000 ; 填写栈顶地址（STM32F401RE的SRAM顶端）

DCD Reset\_Handler ; 复位处理函数标签

DCD Fault\_Handler ; NMI异常

DCD Fault\_Handler ; HardFault硬件错误

DCD Fault\_Handler ; MemManage

DCD Fault\_Handler ; BusFault

DCD Fault\_Handler ; UsageFault

DCD 0 ; 保留，填0

DCD 0 ;

DCD 0 ;

DCD 0 ;

DCD Fault\_Handler ; SVC

DCD Fault\_Handler ; DebugMon

DCD 0 ; 保留

DCD Fault\_Handler ; PendSV

DCD Fault\_Handler ; SysTick系统滴答定时器异常

AREA CODE\_SEG, CODE, READONLY ; 开启程序代码段

EXPORT Reset\_Handler ; 对外暴露Reset\_Handler标签，让向量表可以引用该符号

Reset\_Handler ; 定义Reset\_Handler复位入口标签

;=====开启GPIOA的时钟=====

LDR R0, =0x40023830 ; 将RCC\_AHB1ENR绝对物理地址（RCC起始地址0x40023800

LDR R1, [R0] ; 将寄存器R0指向的寄存器的存储值读取到寄存器R1中

ORR R1, R1, #1 ; 将寄存器R1的bit0置1，使能GPIOA的时钟

STR R1, [R0] ; 将修改后的值写回R0指向的寄存器RCC\_AHB1ENR

;=====配置PA5为通用推挽输出模式=====

LDR R0, =0x40020000 ; 将GPIOA\_MODER绝对物理地址（GPIOA起始地址0x400200

LDR R1, [R0] ; 将寄存器R0指向的寄存器的存储值读取到寄存器R1中

AND R1, R1, ~(0x3 << 10) ; 将寄存器R1的bit10、bit11置0

ORR R1, R1, #(0x1 << 10) ; 将寄存器R1的bit10置1

STR R1, [R0] ; 将修改后的值写回R0指向的寄存器GPIOA\_MODER

LDR R0, =0x40020004 ; 将GPIOA\_OTYPER绝对物理地址（GPIOA起始地址0x40020

LDR R1, [R0] ; 将寄存器R0指向的寄存器的存储值读取到寄存器R1中

AND R1, R1, ~(0x1 << 5) ; 将寄存器R1的bit5置0

STR R1, [R0] ; 将修改后的值写回R0指向的寄存器GPIOA\_OTYPER

LDR R0, =0x40020008 ; 将GPIOA\_OSPEEDR绝对物理地址（GPIOA起始地址0x4002

LDR R1, [R0] ; 将寄存器R0指向的寄存器的存储值读取到寄存器R1中

```

AND          R1, R1, #(~(0x3 << 10))      ; 将寄存器R1的bit10、bit11置0
STR          R1, [R0] ; 将修改后的值写回R0指向的寄存器GPIOA_OSPEEDR
LDR          R0, =0x4002000C ; 将GPIOA_PUPDR绝对物理地址（GPIOA起始地址0x400200
LDR          R1, [R0] ; 将寄存器R0指向的寄存器的存储值读取到寄存器R1中
AND          R1, R1, #(~(0x3 << 10))      ; 将寄存器R1的bit10、bit11置0
STR          R1, [R0] ; 将修改后的值写回R0指向的寄存器GPIOA_PUPDR

;=====主循环=====
main_loop    ; 主循环标签

LDR          R0, =0x40020018 ; 将GPIOA_BSRR绝对物理地址（GPIOA起始地址0x4002000
LDR          R1, =0x00000020 ; BSRR低16位：置位PA5
STR          R1, [R0] ; 将修改后的值写回R0指向的寄存器GPIOA_BSRR

BL           delay_sub ; 跳转至延时子程序

LDR          R0, =0x40020018 ; 将GPIOA_BSRR绝对物理地址（GPIOA起始地址0x4002000
LDR          R1, =0x00200000 ; BSRR高16位：复位PA5
STR          R1, [R0] ; 将修改后的值写回R0指向的寄存器GPIOA_BSRR

BL           delay_sub ; 跳转至延时子程序

B           main_loop ; 跳转至主循环入口

;=====子程序=====
delay_sub    ; 延时子程序标签
LDR          R2, =0x250000 ; 选取R2作为循环计数器，0x500000作为计数器初始值
delay_inner  ; 循环内部标签
SUB          R2, R2, #1 ; 计数器执行自减操作
CMP          R2, #0 ; 判断R2是否为0，将条件标志位置为R2 - 0
BNE         delay_inner ; 若条件标志位不为0则跳转至delay_inner
BX           LR ; 跳转回主循环

Fault_Handler ; 异常处理标签
B           Fault_Handler ; 无条件跳转至Fault_Handler，形成死循环

END

```

- 注意：标签必须顶格，汇编指令必须有空格缩进

1. Cortex-M4上有两个栈指针，分别是MSP（主栈指针）、PSP（进程栈指针），上电复位默认使用MSP，本项目中只使用MSP [↩](#)